

The design of GaugeTheory/

Jinzheng Li

Draft for discussion, 22 September 2026; revised 26 September 2026

Text in blue was added after the first draft: sums of contents (Nathaneal, 23 September) and invariant reduction (24 September).

1 Purpose

Physlib/ClassicalFieldTheory/GaugeTheory/ is the symbolic framework for the local content of a gauge theory: the algebra in which a Lagrangian is written, the gauge and Lorentz actions on it, and the invariants of a given mass dimension. Fields enter only through their jets at one point, the formal symbols $\partial_s \psi_\alpha$.

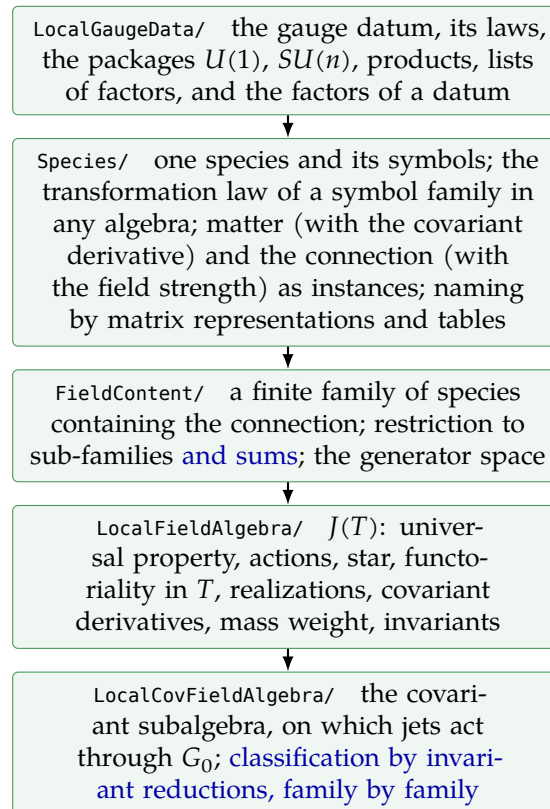
One rule organises the folder: **every layer takes a datum and returns constructions and theorems; a concrete theory contributes only its data**. The Standard Model is a list of gauge factors and a table of fields, and everything else about it is inherited. This note records the intended structure. The present tree differs from it (§6), and the open choices are in §5.

2 The idea

- **Gauge jets.** A gauge transformation is its jet U in a jet group G_J : value $\text{eval } U \in G_0$, Lie algebra $\mathfrak{g}_J$, derivatives, adjoint action, Maurer–Cartan form $\omega_\mu(U) = i(\partial_\mu U)U^{-1}$. These maps and their laws are the gauge datum; nothing downstream knows which group it is.
- **Species.** A species is a finite-dimensional V with a Lorentz representation, a parity, a mass weight, and an affine action of G_J on $\text{JetRing} \otimes V$, a linear part ℓ and a translation t with $t(UV) = t(U) + \ell(U)t(V)$. Matter has $t = 0$ and ℓ fibrewise, generated by an infinitesimal action of $\mathfrak{g}$. The connection has $\ell = \text{Ad}$, $t = \omega$.
- **The algebra.** A content T is a finite family of species containing the connection. $J(T)$ is the free graded-commutative algebra on the symbols of all species. Every action is defined on generators and extended by the universal property; the translation part gives the inhomogeneous term. Reality is a star structure. An inclusion of contents $S \subseteq T$ gives an injective equivariant map $J(S) \rightarrow J(T)$ with a retraction: sectors are its images, and the pure gauge theory is the connection-only content. **Contents also add: $F + G$ is the union of the two families glued along the connection, and $J(F + G)$ is the graded tensor product of $J(F)$ and $J(G)$ over the connection-only algebra, graded by which summands a word touches; both actions preserve the grading.**
- **Invariants.** Mass weight is twice the mass dimension (derivative 2, scalar 2, Weyl fermion 3, connection 2); the dimension-four Lagrangian is the gauge and Lorentz invariants of weight at most 8. On the covariant subalgebra jets act through G_0 , so classification is ordinary invariant theory, sector by sector. **It is done one family of terms at a time: a *reduction* `ReducesInvariantsTo` $\sigma \ V \ W$ says that for every σ -stable submodule S , an invariant of $V \sqcup S$**

lies in $W \sqcup S$. Reductions compose, combine over finite joins and transport along maps of index sets, and the family σ is any family of linear maps, so gauge and Lorentz invariance are one invariance for the disjoint union of the two groups. A classification of one family is a reduction of it to the span of its allowed contractions, proved in the quotient by the remainder. When the pure and mixed pieces span the relevant weight space and satisfy the stability and reduction hypotheses, their classifications combine. Extending a classified theory requires the new-field and mixed classifications; only if the added sector is already classified does just the mixed calculation remain.

3 The layers



Imports go down the line only. Each layer exposes one structure as its datum, `LocalGaugeData`, `Species`, `FieldContent`, and proves everything for an arbitrary instance. Two points (now three) decide the shape:

- **One species notion.** Matter fields and the connection are both species, so the jet component space, the transformation law of a symbol family and the realization notion are stated once. The factors of a gauge datum belong to the gauge layer, not to matter.
- **One algebra tower.** The pure gauge theory is J of the connection-only content, and the sectors of $J(T)$ are images of sub-contents. Nothing is built twice and identified afterwards.
- **One classification mechanism.** The relation `ReducesInvariantsTo` of `Mathematics/InvariantReduction` is general mathematics and stays there. The algebra layer supplies what every theory needs to use it: the gauge and Lorentz actions as one family indexed by $G_0 \sqcup SL(2, \mathbb{C})$, the reduction of a factor's invariance to that family, and the stability of symbol ranges. A classification theorem for one index law (a colour trace, an isospin contraction, a Lorentz scalar) is stated as a reduction modulo an arbitrary stable submodule, so it can be applied inside any larger expression and to any extension.

Each layer is described below by its datum, what it derives, its instances and its interface.

3.1 LocalGaugeData/: the gauge datum

The question it answers. A Lagrangian density at a point x depends on the fields and finitely many of their derivatives at x . So when we ask whether it is gauge invariant, a gauge transformation $g(x)$ matters only through its Taylor series at that point, its *jet*. This layer describes these jets and nothing else. It does not mention fields or Lagrangians; every later layer takes its datum as input.

An example first. For $U(1)$, a gauge transformation is $u(x) = e^{i\theta(x)}$. Its jet is the power series of u in the coordinates $x_0, \dots, x_3$, a unitary element of $\mathbb{C}[[x]]$. Its value at the base point is $u(0) \in U(1)$. Its Maurer–Cartan form $\omega_\mu = i(\partial_\mu u)u^{-1} = -\partial_\mu \theta$ is exactly the shift of the gauge field, $A_\mu \mapsto A_\mu + \omega_\mu$. For $SU(n)$ everything is the same with $n \times n$ matrices of power series, and $\omega_\mu = i(\partial_\mu U)U^{-1}$ is a traceless hermitian matrix.

The datum. LocalGaugeData $G_0 \ \mathfrak{g} \ G_J \ \mathfrak{g}_J$ abstracts the example. It has four types: the gauge group G_0 and its Lie algebra $\mathfrak{g}$, and their jet versions G_J and $\mathfrak{g}_J$. It has the maps a transformation law uses:

- evaluation at the base point and constant jets, $G_J \hookrightarrow G_0$ and $\mathfrak{g}_J \hookrightarrow \mathfrak{g}$;
- derivatives ∂_μ and multiplication by coordinates x_μ on $\mathfrak{g}_J$;
- the adjoint action Ad_U of a gauge jet on Lie algebra jets;
- the Maurer–Cartan form $\omega_\mu(U)$.

It also has the rules these satisfy in the example, such as $\omega(UV) = \omega(U) + \text{Ad}_U \omega(V)$ and $\partial_\mu \omega_\nu - \partial_\nu \omega_\mu + [\omega_\mu, \omega_\nu] = 0$. Everything later is proved from these rules alone, so it holds for every gauge group at once.

What is proved from the rules. Three tools that later layers need:

- how Ad_U acts on Taylor coefficients (a Leibniz formula), which is how a gauge jet moves the derivatives of a field;
- the *symmetrised* Maurer–Cartan data, a smaller set of numbers from which all Taylor coefficients of ω can be recovered;
- the jets that agree with the identity up to order n ; these form normal subgroups, and such a jet only shifts the first n derivatives of the gauge field.

Two extra properties. The rules do not say that G_J really consists of Taylor series, so two properties are stated separately. *Faithful*: a jet is determined by its Taylor coefficients. *Free*: every choice of Taylor coefficients comes from some jet. *Free* is what later shows that the gauge field enters an invariant only through its field strength: gauge jets are rich enough to remove every other combination of derivatives of A_μ .

Where concrete groups come from. A *matrix presentation* (MatrixJets) says that the four types sit inside matrices and every map is the matrix one. From it the rules, *Faithful* and a test for *Free* are proved once, so $SU(n)$ (`su n`) and $U(1)$ (`u1`, as 1×1 matrices) are obtained by filling in a presentation. Products (`prod`) and a list of factors (`ofFactors`) then give any group such as $SU(3) \times SU(2) \times U(1)$, written [`.SU 3, .SU 2, .U1`].

Factors and morphisms. Charges are given one factor at a time, for example “a doublet of $SU(2)$ with hypercharge -3 ”. A *factor* (`U1Factor`, `SUFactor`) records how the gauge jets project onto one $U(1)$ or $SU(n)$. A *morphism* (`Hom`) is a map between two data that respects all the maps above, and a factor of the target pulls back along it. The projections of a product are morphisms, which is how the factors of $SU(3) \times SU(2) \times U(1)$ are obtained from those of its pieces.

Conventions. The datum is always passed explicitly, never found by instance search, because the four types do not determine it. The layer imports nothing from the layers above it.

Species: requirements from the classification

Fields and symbols. Keep the value-space representation distinct from the dual action on field symbols, including their conjugates. Parity and mass weight belong to the species; bases and particular fundamental, conjugate or adjoint representations are inputs to specific classifiers, not requirements on every species.

Compatibility. Keep the conditions behind covariance visible. The current `GaugeLorentzCompatible` supplies Lorentz stability of covariant expressions; `PureJetsActTrivially` makes their jet action factor through evaluation. Whether these become consequences of the new species definition or remain hypotheses must be explicit. The converse statement, that jet-invariant expressions lie in the covariant algebra, uses gauge freeness and is a separate result.

What the assembly needs. Each sector supplies a reduction, containment of its classified span in the source, and pointwise fixedness of that span. These combine without sector independence. Covariance and weight laws should be derived from species data and exposed to realizations. The established classification spans complex formal expressions; it imposes no reality condition and takes no quotient by total derivatives or equations of motion.

Migration boundary. Begin the SM realization migration once the species-to-field-content interface, the table compiler and their algebra/realization interfaces are stable. Not every species theorem must be finished first. The shared reduction mathematics is independent of this packaging; duplicate SM interfaces can be removed as their consumers migrate.

4 What a model sees

A model is a card and nothing else:

```
abbrev gauge : List FactorSpec := [.SU 3, .SU 2, .U1]
abbrev leptonDoublet : SpeciesData factors := (.L, .singlet, .fund, -3)
...
noncomputable def content : FieldContent gaugeData := table.toFieldContent
```

Species of the model are abbreviations of the value spaces the card produces, and theorems are stated in the card and the generic vocabulary alone, for example that the gauge and Lorentz invariants of content of weight at most 4 are spanned by 1 and $H^\dagger H$.

5 Questions for discussion

1. **One species structure or two.** The affine law costs a translation field that is zero for all matter. Is the uniformity worth it, or should `MatterField` and the connection stay separate with the

symbol-family law stated twice?

2. **Real or complex tower.** The connection's algebra is real today; the single tower is complex with a star. Does anything downstream need the real algebra as an object?
3. **The connection in the content.** Explicit member of the family (restriction becomes ordinary) or implicit through the gauge datum (as today)? [Explicit forces a sum \$F + G\$ to be glued along the connection; implicit allows a plain disjoint union.](#)
4. **Sectors by functoriality.** Restriction along $S \subseteq T$ fixes how species are indexed. Is that acceptable to the existing sector proofs? [Images of sub-contents give the "at most \$S\$ " subalgebras, but those proofs use the "exactly \$S\$ " pieces, and a sum has a mixed part in neither image. Should the grading by sub-families be part of this layer?](#)
5. **Names and order of work.** Species, FieldContent, content against the current names; and which of the steps below, if any, should precede the remaining Standard Model migration?

6 Distance from the current tree

current	intended
LocalGaugeData/	the same, plus the factor types now in MatterField/
MatterField/; GaugeBoson/Basic, GaugeBoson/	MatrixRep/
Realization/	Species/
GaugeBoson/LocalGaugeFieldAlgebra/, GaugeBoson/LocalGaugeCovFieldAlgebra/	the two algebra layers at the connection-only content
GaugeFieldData/ LocalFieldAlgebra/, LocalCovFieldAlgebra/	FieldContent/, with the connection explicit
Mathematics/InvariantReduction; StandardModel/InvariantReduction	the same, sectors derived from functoriality the same; the combined family, the factor reductions and the range stabilities move into LocalCovFieldAlgebra/, stated for ofFactors

In the order they should be removed:

1. LocalGaugeData/ imports matter (the factor types, and TransformsIn, InfinitesimalAction importing the covariant derivative). Move the factor types down and the two files into the species layer.
2. Matter and the connection have differently shaped APIs, TransformsIn for matter and GaugeAlgebraRealization for the connection. Introduce the species structure with both as instances.
3. Sectors are built by hand and the gauge sector identified with a separate algebra (four GaugeSector files). Prove functoriality and derive them.
4. The connection has its own real algebra tower of about 3300 lines. Replace it by the connection-only content, last, because the Standard Model gauge-sector classification is built on it.

Steps 1 and 2 are cleanup; 3 and 4 change shared conventions and need their owners.

7 Progress

[Kept up to date as the layers are reorganised. Each entry says what was changed and what the layer now is, so that a reviewer can check the commits against the intent above.](#)

Changed. The layer imported matter in two ways; both are gone. The factor types `U1Factor`, `SUFactor`, `Factor`, `Factors` moved down from the matrix-representation and table files into a new `Factor.lean`, since their fields mention only gauge jets, the gauge algebra and the jet ring. `TransformsIn` and `InfinitesimalAction`, which are statements about a representation on $\text{JetRing} \otimes V$, moved up to `MatterField/`. The carriers of a list of factors were renamed from `Factors.G` to `OfFactors.G`, so that `Factors` names only the type. The duplication between `U1.lean` and `SU/Basic.lean` was removed by a new `MatrixJets.lean`: a presentation of gauge jets by matrices of jets (injective maps of the four carriers into $\kappa \times \kappa$ matrices, and what each structure map is in matrices), from which the laws of `LocalGaugeData`, faithfulness and the canonical `su`-type factor are proved once; the matrix identities over the jet ring it needs went to `Relativity/JetRing/Matrix`. `su n` and `u1` are now instances, `u1` reading a scalar as a 1×1 matrix. Eight lemmas referenced nowhere, a stale TODO and a helper definition were removed, and the TODO on the recursive carriers was replaced by the reason for the recursion.

Now. Eleven files, about 3 500 lines, importing nothing outside the layer. `Basic` (the structure, its laws, the iterated derivative, `Faithful`); `AdjointCoeff` (the Taylor–Leibniz theorem); `MaurerCartan` (the symmetrised form and what determines the form); `Truncation` (the filtration, pure jets, `Free`); `Factor` (the factor types); `MatrixJets` (the presentation by matrices); `SU/Algebra`, `SU/Basic`, `U1` (the concrete packages as instances); `Prod` (products and lifting factors); `OfFactors` (a list of factor symbols, its carriers and canonical factors).

Follow-up (Joseph, 25 September). Freeness is now also proved once for a matrix presentation: `MatrixJets.free` reduces `Free` to three conditions on the carriers (Lie algebra jets contain every matrix with Taylor data in $\mathfrak{g}$, they are hermitian, and the gauge jets contain the unitary Euler transport of every Lie algebra jet), using the Euler-transport results on matrices of jets. The Standard Model freeness proof becomes an instance and shrank by about 400 lines. The layer is now about 3 600 lines, still importing nothing outside itself.

Not done. `U1.lean` and `SU/Basic.lean` still each define their structure maps by hand (about 150 lines each); a further step would define the carriers themselves as subtypes of matrices and take the maps from `MatrixJets`, but that changes the carrier types the Standard Model files unfold to, so it waits for the hand-built Standard Model code to go.